

EE200: Signals, Systems and Networks

Instructor: Dr. Tushar Sandhan
Topic: Course Project

Department: EE
Marks: 25%

Instructions

- You have to submit each question separately on mookit in the format given in the appendix here
- You have to form a group of maximum 2 members
- In case you could not form a group, you may submit project as an individual
- Marks for each question are given in brackets
- Corresponding required data for the project: <https://home.iitk.ac.in/~sandhan/>

Q1A. Frequency Forensics

‘The Ghost Signal’ [3%]

In intelligence and surveillance systems, information is often transmitted through hostile environments where signals may be intentionally corrupted to prevent unauthorized access. Imagine that a reconnaissance satellite intercepts a grayscale image suspected to contain critical information. Unfortunately, the received image appears heavily distorted by a repetitive pattern, making the original content nearly impossible to recognize. Intelligence analysts believe that the image contains an important question whose answer may reveal the next stage of the mission. However, before the question can be answered, it must first be recovered from the corruption. To an observer in the spatial domain, the image looks corrupted beyond repair. However, a signal processing engineer knows that periodic interference leaves unique fingerprints in the frequency domain.

An image $I(x, y)$ is a finite two-dimensional discrete signal containing intensity variations along both spatial directions. Since periodic disturbances correspond to specific frequency components, analyzing the image in the Fourier domain may reveal the source of corruption. By transforming the image into the frequency domain, can you identify signatures of the unwanted interference? What mathematical equations govern the two-dimensional Discrete Fourier Transform (DFT) and its inverse? If you plot the magnitude spectrum of the given image in both linear and dB scales, where are the dominant spectral components located? Is the spectrum naturally centered, and if not, how can it be shifted to place the lower frequencies at the center of the transform?

The interference has been deliberately introduced in such a way that only a few frequency components are responsible for the corruption. Can you locate these components in the spectrum? How do they differ from the frequencies associ-

ated with the actual image content? Design a suitable frequency-domain system capable of suppressing the unwanted frequencies while preserving the useful information. Carefully analyze how periodic noise manifests itself in the Fourier domain and justify the frequencies selected for removal.

After filtering, reconstruct the image using the inverse Fourier transform. Has the hidden message been successfully recovered? Compare the corrupted image, its frequency spectrum, the filtered spectrum, and the recovered image. How does the choice of filter parameters affect the quality of reconstruction? Does removing more frequencies always improve the result, or can it also destroy useful information? Discuss your observations and comment on practical applications of frequency-domain image restoration in communication systems, remote sensing, surveillance and medical imaging. This is known as a Frequency-Domain Image Recovery System. Use the `ghost_signal.input` image as an input to this system.

Q1B. Digital Detective

‘Missing Boundaries’ [2%]

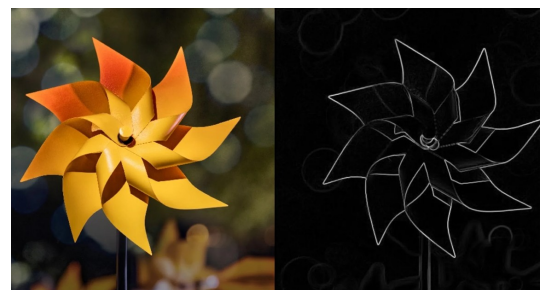


Figure 1: Edge detector system: input & output

Humans can effortlessly recognize faces, vehicles, roads, handwritten text and everyday objects, often within a fraction of a second. Surprisingly, much of this ability comes from identifying boundaries where intensity changes abruptly. These boundaries, known as edges, contain a signifi-

cant portion of the structural information present in an image. Modern technologies such as autonomous vehicles, medical imaging systems, facial recognition software and industrial inspection tools all rely on the ability to detect and analyze edges accurately. Image $I(x, y)$ is a finite two-dimensional discrete signal whose intensity varies in both spatial directions. Abrupt intensity transitions often correspond to object boundaries and therefore contain valuable information about the scene. What mathematical operation can be used to identify regions where intensity changes rapidly? How are derivatives and gradients related to edge detection? Explore the concepts behind the Sobel filter, which is used for 2D convolution operation on images to detect edges.

Using the given image, apply the Sobel kernel to reveal the hidden boundaries within the scene. How does noise affect the detected boundaries, and what role does image smoothing play before edge extraction? Can weak edges be preserved without introducing excessive noise? Use `missing_boundaries_input` image for your complete analysis.

Q2. The Midnight Episode

‘Catching the Arrhythmia’ [7.5%]

Maya is interning this summer at a cardiology lab. One of the patients was sent home wearing a Holter monitor, a small portable device that records the heart’s electrical activity (an ECG) continuously for many hours while the patient goes about ordinary life. When the patient came back, his recording was hours long, and his cardiologist suspects that at some point during the night his heart slipped into an arrhythmia: an episode where the steady, regular heartbeat becomes irregular. Scanning the whole recording manually is slow and error-prone.

Maya recalls two pictures from her Signals & Systems class (Figure 2). In a healthy heart (Figure 2a) the ECG is almost periodic, the same shape repeats at steady intervals, like a metronome: a small bump (the P wave), a tall sharp spike (the QRS complex), and another bump (the T wave). During an arrhythmia (Figure 2b) this regularity falls apart: beats arrive too early or too late, spikes change shape or even point the wrong way, and the clean repetition is lost.

Maya had completed EE200, so she could devise an elegant plan. She decided rather than read the whole signal, she will teach the computer to recognize one healthy heartbeat, then convolve it with the recording and check, stretch by stretch, whether the signal still looks like a healthy beat. Where the match breaks down, the episode has begun.

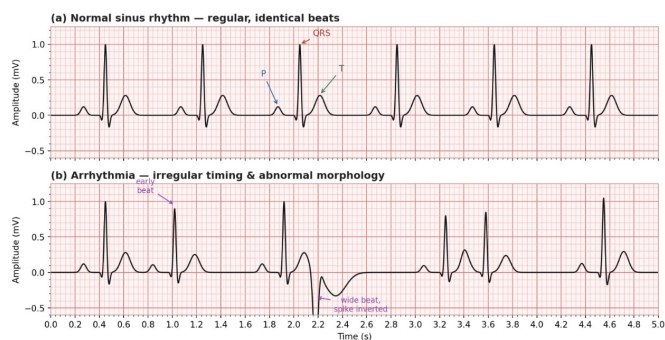


Figure 2: ECG: Electro-Cardio-Grams

The recording. The clip Maya is studying is a discrete-time signal $x[n]$ sampled at $f_s = 250$ Hz, and it is having $N = 5000$ samples. In the early (healthy) portion, the heart beats with metronome regularity: one full beat {P, QRS, T} arrives at every 0.8 seconds.

- (a) Reading the signal: [0.75%]
 - (i) How many seconds long is the clip?
 - (ii) In the healthy stretch, what is the patient’s heart rate in beats per minute, and how many samples does one healthy beat occupy?
 - (iii) Treating the healthy ECG as a periodic signal, what is its fundamental frequency f_0 (in Hz)?
- (b) Healthy heart in frequency domain: [0.75%]
 - (i) As the healthy ECG is (nearly) periodic, what does its magnitude spectrum $|X(f)|$ look like, a smooth continuous curve, or something else?
 - (ii) The QRS spike is sharp and narrow; the P and T waves are broad and smooth. Which of these is responsible for the higher-frequency content of the ECG, and why?
 - (iii) Suppose the patient’s heart rate rises to 150 bpm but stays perfectly regular. What happens to f_0 , and to the spacing between the components in the spectrum?
- (c) Cutting a heartbeat (windowing): [1.0%]

To build her template, Maya multiplies the signal by a rectangular window, a function equal to 1 inside some interval $[n_1, n_2]$ and 0 outside, to snip out a segment.

 - (i) To capture exactly one full beat, how wide (in samples) should the window be, and roughly where in the recording should she place it?
 - (ii) She first tries a window only 80 samples wide, then one 600 samples wide. For the purpose of making a clean ‘one healthy beat’ template, explain what goes wrong in each case.

- (iii) In class you saw that a short window gives sharp time resolution but poor frequency resolution, while a long window does the reverse. In one or two sentences, relate that same trade-off to Maya's choice here: why isn't 'make the window as short as possible' automatically the best idea?

- (d) Match the template (correlation): [1.5%]

Let the template (one healthy beat) be $t[k]$ for $k = 0, 1, \dots, L - 1$. Sliding it to start at position m , Maya compares it with the segment $x[m], x[m + 1], \dots, x[m + L - 1]$ using a normalized correlation score as,

$$\rho(m) = \frac{\sum_k t[k] x[m + k]}{\|t\| \|x_m\|}$$

where $\|\cdot\|$ is the energy (root-sum-of-squares) of a segment. (You do not need to compute this by hand, just reason about it.) Intuitively, $\rho(m)$ measures how similar in shape the template and the segment are.

- (i) What range of values can $\rho(m)$ take, and what value signals a near-perfect match?
- (ii) Real ECG amplitude drifts slowly up and down over time (this drift is called baseline wander), and beats vary in size. Why is the normalization (dividing by the energies) important? In particular, what would happen to an unnormalized score $\sum_k t[k] x[m + k]$ if a perfectly healthy beat happened to be twice as tall as the template?
- (iii) During the arrhythmia, one abnormal beat is inverted (flipped upside-down) relative to the template. Roughly what value of ρ would you expect for it, and why does that make inverted beats especially easy to flag?
- (e) Onset detection & the spectrogram: [1.5%]
Maya computes $\rho(m)$ at every position across the 20-second clip and plots ρ against time. In the healthy region, it reaches 1; somewhere it drops and sometimes strongly negative.
 - (i) Propose a simple rule Maya could use to declare an onset time for the arrhythmia. Then discuss the trade-off in setting her threshold too high versus too low.
 - (ii) A labmate suggests a second, independent view: a spectrogram of the whole clip (slide a window along the signal, take the DFT of each windowed piece, and stack the results as columns over time). Describe how the spectrogram would look different in the healthy region versus the arrhythmia region.

- (iii) The correlation plot and the spectrogram might disagree slightly on the exact onset time. Explain why, and mention which one you would trust to pinpoint the single moment a bad beat first appears. (Hint: think back to the trade-off in part (c).)

- (f) sampling & aliasing: [0.5%]

To make wearable's battery last longer, an engineer proposes recording at only $f'_s = 50$ Hz instead of 250 Hz. The clinically important sharp features of the QRS complex contain frequency content up to about 40 Hz.

- (i) By the Nyquist theorem, what minimum rate is needed to capture content up to 40 Hz without aliasing?
- (ii) In plain words, what would aliasing do to the sharp QRS spikes at 50 Hz, and why is that dangerous for Maya's detector?
- (iii) If the team really must lower the sampling rate, what is the fix, and what is the unavoidable cost of it for an ECG?

- (g) Prototyping the detector in code: [1.5%]

Maya wants to test her logic on her laptop. She has exported the 5000 sample recording into a file called `patient.ecg.npy`, and her clean 200-sample template into `template.npy`. Write a Python code `find_onset(ecg_signal, template, threshold)` that:

- (1) Calculates the normalized correlation score ρ for the signal beat-by-beat (i.e., jumping forward by the length of the template each time, not sample-by-sample).
- (2) Identifies and returns the index m of the very first beat where the correlation drops strictly below the provided `threshold` (assume a threshold of 0.5). If it is never breached, return -1.
- (h) Visualizing the Spectrogram: [0.5%]
Maya wants to visually verify the arrhythmia using a spectrogram before running the correlation script. Assume the 5000-sample recording is loaded as a 1D NumPy array called `ecg_signal`. Using the `scipy.signal` and `matplotlib.pyplot` libraries, write a brief Python snippet to compute and plot the spectrogram of the recording.
Crucial step: Based on your reasoning in part (e), choose a specific window length (`nperseg`) in samples that will successfully reveal the steady horizontal harmonic bands in the healthy region. State the window length you chose and briefly comment (1 sentence) on why you picked that exact number.

Q3A. Sonic Signatures

‘Magical Mystery Tune’ [7.5%]

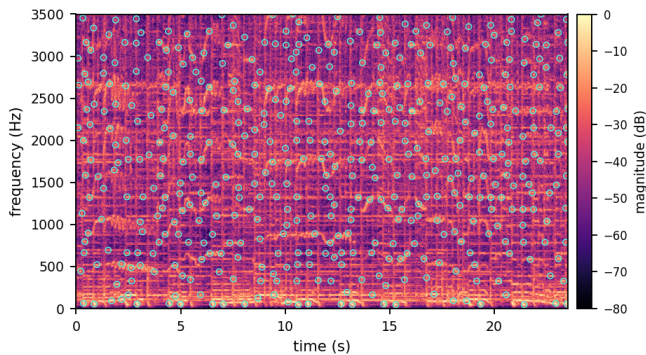


Figure 3: A spectrogram of a short clip, with the strongest time-frequency peaks $\rightarrow$ its *constellation* (circled)

You are in a noisy café when a song you cannot name starts playing. You ask your music quizzier friend who also has no idea. So you hold up your phone, and within seconds it tells you the title, recognised from a few seconds of sound, over all the chatter. Remarkably, it never compares the actual audio waveforms. Instead it turns the music into a picture of which frequencies are present at each instant, and keeps only a sparse set of standout points from that picture as the song’s *fingerprint*. In this question you will build a small version of this identifier. First, see why a single Fourier transform will not do. Plot the DFT magnitude of an entire song: you can read off *which* frequencies it contains, but not *when* they were played, all sense of timing is gone. What we really want is a way to watch the frequency content *change* as the song plays, and that is exactly what a *spectrogram* gives us. The idea is simple: slide a short window along the signal and take the Fourier transform of each short slice, then stack those slices side by side into an image. Time runs along the horizontal axis, frequency up the vertical axis, and the brightness of each point shows how strong that frequency is at that moment: so a rising note traces a rising streak, a steady tone a horizontal line. Each slice is nothing more than an ordinary DFT, the Fourier analysis applied to one short piece of the signal at a time. Compute and plot the spectrogram of a provided song. Now experiment a little: redo it with a short window and with a long one, and describe what you observe about the resolution in time versus in frequency.

Next, build the fingerprint. From the spectrogram keep only the strongest *peaks*, the local maxima that stand out from their surroundings (the circled points in Figure 3), and plot this ‘constellation’ of points. Pair the nearby peaks together into compact hashes (two frequencies and the time gap between them) and store them in a database

for several provided songs. To identify a new clip, fingerprint it the same way and, for each song, look at the time offsets at which its hashes match: a true match lines them all up at a single offset, while a wrong song gives only scattered, random matches. Output the name of the matched song. Then repeat the matching using single peaks on their own instead of pairs, and compare what you get, explain why joining two peaks into a single fingerprint makes a correct match so much more decisive.

Now see how robust your identifier is. Add increasing amounts of noise to a query and find how far you can push it before recognition fails; then shift the whole clip up in pitch by a little (or stretch it slightly in time) and try again. Describe what you see in each case and explain why; in particular, why a small pitch shift can defeat the identifier even though the song still sounds the same to you. Suggest one change that would make the system more robust. Present your spectrograms, plots, and written observations in the report.

Q3B. Signals to Softwares

‘Zapptain America’ [5%]

Wrap your identifier in a simple interactive app, that a user can run: it should index the provided songs into a database, accept a new query clip, and display the recognised song, ideally showing the intermediate steps (the spectrogram, the constellation of peaks, and the offset histogram that decides the match). Refer to the provided demo video showcasing expected functionality for a better idea.

■

Appendix

Submission and Evaluation Guidelines for Q1A and Q1B

- **Evaluation Scheme:** 40% marks will be awarded for the final results, analysis and correctness of the output, while 60% marks will be awarded for the methodology, implementation process, explanations, justification of approaches and the quality of the project report.
- **Submission Requirements:** Submit a Jupyter Notebook (.ipynb) containing the complete implementation and answers to all the sub-questions, along with a project report (PDF) containing the methodology, plots, observations and final results.
- **Dataset Usage Policy:** Only the provided images must be used as inputs for the assignment, and all results must be obtained from the given dataset. The use of AI-generated or externally generated images is strictly prohibited and will attract a strict penalty.
- **Report Expectations:** The report should include relevant plots, intermediate observations, comparisons, discussions and final conclusions. Marks will be awarded for both the quality of the results and the reasoning used to obtain them, not just the final output.

Submission and Evaluation Guidelines for Q2 (all parts)

- **Submission Requirements:** Submit a Jupyter Notebook (.ipynb) containing the complete implementation and answers to all the sub-questions, along with a project report (PDF) containing the explanations, plots, observations and final results.
- **Clarity of Responses:** Responses should be concise and directly address the core problem. Please avoid extraneous details and overly lengthy explanations.
- **Dataset and Reporting:** Analysis must be conducted exclusively using the provided dataset. Additionally, all generated plots, figures, and explanations must be embedded directly within the final report file.

Submission and Evaluation Guidelines for Q3 (A) and Q3 (B)

- **Submission Requirements:** For **Q3 (A)**, submit your report as a PDF containing the spectrograms, experiments, observations and explanations. For **Q3 (B)**, *deploy* your application and submit a link to the live app in the same PDF, so that we can open and test it directly: both the single-clip mode and the batch mode, exactly as a user would use it; also include a link to its source code. Hosting is free and quick on Streamlit Community Cloud (or a similar service); make sure the indexed song database ships with the deployed app so it works immediately. Also, submit all your code in a zip file.
- **What to Build → Two Modes:** (i) a *single-clip mode* that identifies one uploaded query clip and displays (at the very least) the intermediate steps: the spectrogram, the constellation of peaks, and the offset histogram that decides the match; and (ii) a *batch mode* that accepts a set of query clips and writes a file `results.csv` with exactly two columns, `filename`, `prediction`, where `prediction` is the matched song's filename without extension. The batch format must be followed exactly, as it will be evaluated automatically. Note: If the submitted link does not work then you will be awarded 0 marks for Q3 (B).
- **Song Database:** Download the provided song library from the link on the course webpage, index it once, and keep the resulting database. Use the files exactly as provided and do not rename them: a song's filename (without extension) is the label your identifier must output.
- **Tools:** You do not need prior experience building apps; lightweight frameworks such as Streamlit or Gradio let you wrap your functions into a usable app rather easily. You may use any other frameworks you are familiar with provided the single-clip visuals and the exact `results.csv` format are produced.
- **Report Expectations:** Include the spectrograms you computed and, for each experiment: window length; single peaks vs. paired hashes; added noise; pitch shift / time stretch → the relevant plots together with a written explanation of what you observed and why.